

ADVANCED OPERATING SYSTEMS

Lab 1 — Kernel development and debugging

Vittorio Zaccaria

Politecnico di Milano

Overview

What you will build

One kernel module of your own, a reading of a real kernel failure, and three GDB sessions on the running kernel.

- **Part A — your first module.** Build, load and remove the hello starter in a working copy, then make its parameter validated and writable at runtime.
- **Part B — kernel failures.** Read an oops report and locate the failing instruction, the task and the call path.
- **Part C — the kernel under GDB.** Stop the `init_module` system call that loads your module, follow it into your init function and back, then stop a timer callback before it runs.

Each part ends with a **checkpoint**. Answer the questions on each checkpoint slide before moving on.

The lab answers two questions. Loading a module creates no process: so **who runs its code** (A, C)? And what does the kernel tell you **when its own code fails** (B, C)?

Set up the environment

Build the image once (long: it downloads and compiles the kernel):

```
T=amd64 E=full make build-container
```

Then use **one** frontend. Both edit the same repository and use the same kernel, module build, initramfs and QEMU.

Host terminal and LazyVim (exit LazyVim with `:qa`):

```
T=amd64 E=full make dev-up      # once
T=amd64 E=full make dev-build   # after every change
T=amd64 E=full make dev-run
T=amd64 E=full make dev-vi F=modules/lab-1-work/module.c
```

Browser editor: `make vscode-build-container`, then `make vscode-up`; open `http://127.0.0.1:8080` with password `aos-linux-labs` (port busy: `make VSCODE_PORT=8081 vscode-up`). Build with **Terminal → Run Build Task** (`AOS: Build modules and initramfs`); boot with **Terminal → Run Task →** **`AOS: Run QEMU (1 vCPU)`**.

Set up the working module

`modules/lab-1-intro-hello-module` is the published starter: keep it unchanged and work on a copy.

```
# Host
rm -rf modules/lab-1-work
cp -Rf modules/lab-1-intro-hello-module modules/lab-1-work
```

Every `modules/lab-*` directory is built by `make dev-build` (or the browser build task) and installed in the guest as `/modules/<directory>.ko` : yours is `/modules/lab-1-work.ko` . Only `module.c` is compiled, so the module's internal name is `module` : that is the name `rmmod` and `/sys/module` use.

The guest has no disk: its whole root filesystem is an **initramfs** (busybox) generated by the build. A running guest never sees a rebuilt module: exit QEMU with `Ctrl-]` , then `x` , rebuild, boot again.

Part A — A module and its parameters

A1 · Task: build, load and remove the starter

Read the starter (now also in `modules/lab-1-work/module.c`) and find:

- the function that runs at `insmod` and the one that runs at `rmmod`, and how they are registered;
- the parameter `num`: its default value and its permissions;
- the three lines printed by init and the two printed by exit.

Build, boot and load the module with `num=7`. **Before running**, predict which process name init and exit will print: loading a module creates no process.

A1 · Useful APIs

NEED	API
run code at load / unload	`module_init(fn)`, `module_exit(fn)`; init returns 0 or `-errno`
a load-time parameter	`module_param(name, type, perm)`; set with `insmod file.ko name=value`
log a line (read with `dmesg`)	`pr_info(fmt, ...)`; also `pr_err`, `pr_warn`, …
the task executing this code	`current`: name `current->comm`, PID `task_pid_nr(current)`
print a pointer	`%p` hashed; `%px` raw address (only in a test VM); `%pK` per `kptr_restrict`

A parameter with non-zero permissions appears as `/sys/module/<module>/parameters/<name>`. Format specifiers: printk formats.

A1 · Checkpoint

```
# Guest
insmod /modules/lab-1-work.ko num=7
echo status=$?                              # status=0
cat /sys/module/module/parameters/num       # 7
rmmod module
dmesg | grep hello_
```

Example from one run (PIDs and addresses vary):

```
hello_init: comm=insmod pid=76
Address of hello_init = (____ptrval____) (ffffffffc0005000)!
hello_cleanup: comm=rmmod pid=81
```

Questions.

1. Which task ran `hello_init`, and which ran `hello_cleanup`? Did any task run the module's code between the two?

2. Why does `%p` print `(____ptrval____)` and not the address?

A2 · Task: validate and update the parameter

Change `modules/lab-1-work/module.c`:

1. Make `num` readable by everyone and writable by root: permissions `0644`.
2. At the start of the init function, before any other line: if `num` is negative, log `hello_init: invalid num=<num>` and return `-EINVAL`.
3. In the exit function, also log `hello_cleanup: num=<num>`, with the value `num` has when the module is removed.

Keep everything else from A1. Rebuild, boot a new guest and run:

```
# Guest
insmod /modules/lab-1-work.ko num=-1
echo status=$?
cat /proc/modules
insmod /modules/lab-1-work.ko num=7
echo 9 > /sys/module/module/parameters/num
rmmod module
dmesg | grep -E 'hello_(init|cleanup):'
```

A2 · Checkpoint

Expected: the first `insmod` fails with

```
insmod: can't insert '/modules/lab-1-work.ko': invalid parameter
```

then `status=1` and an empty `/proc/modules`. The log (PIDs vary):

```
hello_init: invalid num=-1
hello_init: comm=insmod pid=77
hello_cleanup: comm=rmmod pid=79
hello_cleanup: num=9
```

When init returns a negative error, the load fails: the module is freed and its exit function never runs.

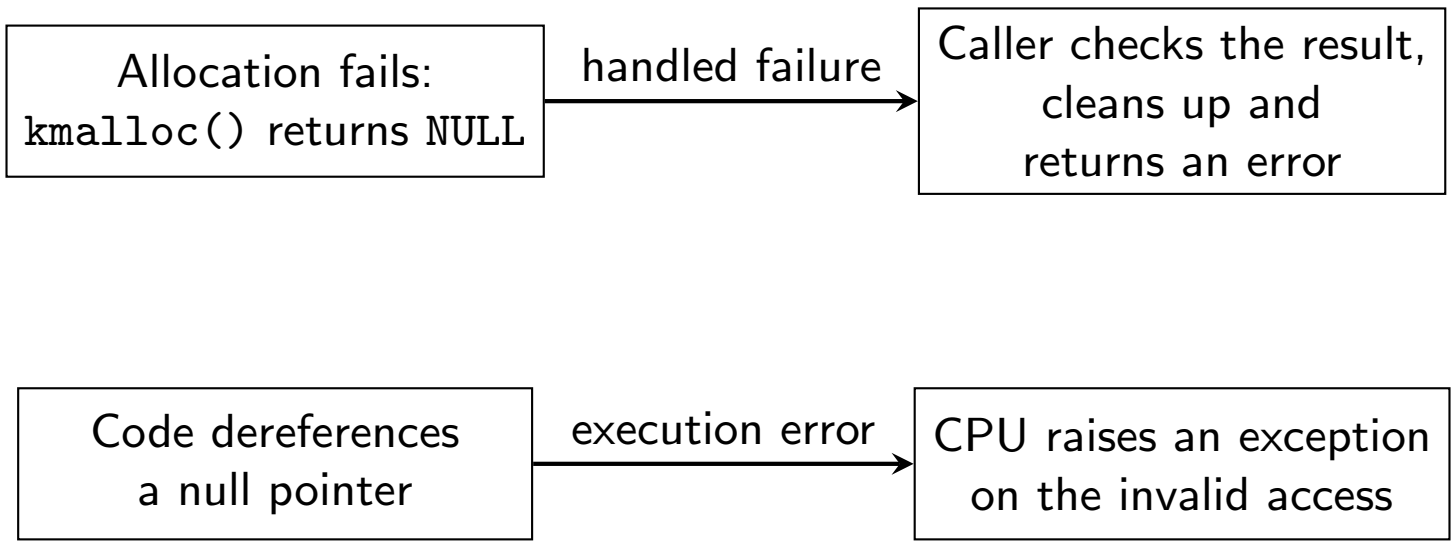
Questions.

1. Was the failed load a kernel failure? Which code decided what `insmod` printed?
2. `num` became 9 without any call into your module. Which task wrote it?

Part B — When kernel code fails

B · An error return is not an oops

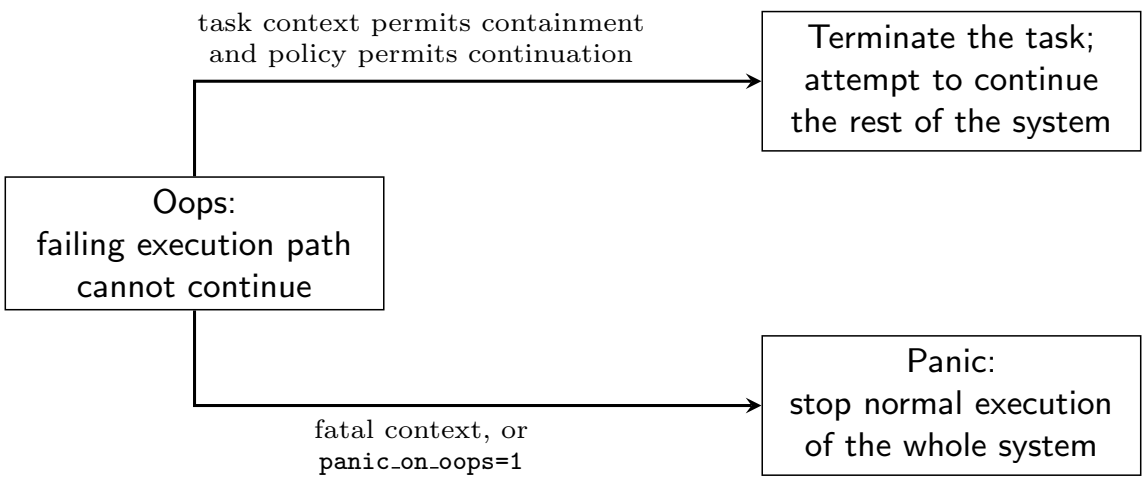
An expected failure is handled through normal control flow. A programming defect can instead make kernel execution invalid.



A failed allocation leads to the second case if the caller dereferences the null result instead of checking it. Not every invalid access is detected immediately: a write to mapped memory may silently corrupt another object.

B · Oops and panic

An **oops** is Linux's handling of a kernel execution error it cannot recover from: it prints a report and abandons the failing path. A **panic** stops the whole system.



- The CPU exception is the hardware event; the oops is the kernel's report.
- In an interrupt there is no calling task to terminate. Continuing after an oops does not guarantee consistent state.
- Some faults have recovery paths (e.g. user-memory copy helpers) and cause no oops.

B1 · Task: read an oops report

Learn where each part of a report is from this annotated excerpt, then answer the checkpoint from the full log.

REPORT EXCERPT

```
BUG: kernel NULL pointer dereference,
address:  0000000000000000
#PF: supervisor write access in kernel mode
Oops:  0002 [#1] PREEMPT SMP NOPTI

CPU: 0 PID: 77 Comm:  insmod
Tainted:  G O 5.16.0-rc1 #1

RIP: 0010:oops_demo_init+0x5/0x1000
[oops_demo]

RSP: 0018:ffffc900001d7cb0 EFLAGS: 00000246
RAX: 0000000000000000 RBX: 0000000000000000
RCX: 0000000000000000

Call Trace:
<TASK>
do_one_initcall+0x44/0x200
[...]
load_module+0x2381/0x26b0
[...]
Modules linked in:  oops_demo(O+)
```

INTERPRETATION

Error and address
A kernel-mode write to address zero.
`#PF` means page fault.

Execution environment
CPU, task, kernel version and taint.

Failing instruction
`RIP`: function + byte offset,
then the containing module.

Register state
Stack pointer, flags and data registers.
Decode the instruction before
assigning a meaning to each value.

Call path
Module initialization was reached
through the module loader.

Module state
`O`: out-of-tree; `+`: being loaded.

B1 · Checkpoint

diagnostics/oops-demo.log is the full report of a module `oops_demo` whose init writes through a null pointer (Linux `5.16.0-rc1`, this QEMU). There are no guest commands: answer from the **full** log, not only from the excerpt.

Questions.

1. Which instruction failed? Give the function, what `+0x5/0x1000` means, and the kind of access (`#PF` lines and `error_code`).

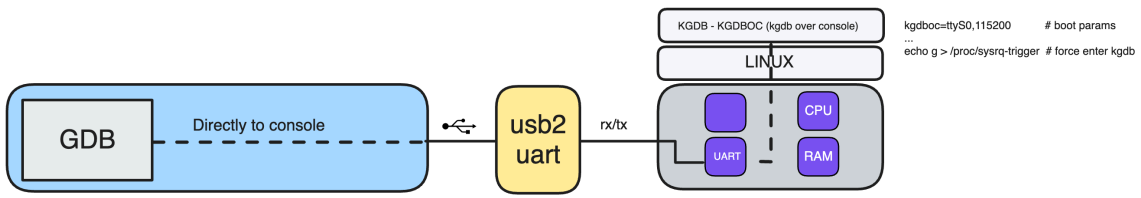
2. Inside the call trace there is a second register dump with `RIP: 0033:` and `ORIG_RAX: 00000000000000af`. Whose registers are these, and which system call do they identify?

3. The log ends with `Killed`. What happened to `insmod`, and did the guest keep running? Would the outcome be the same if the faulting write ran in a timer callback?

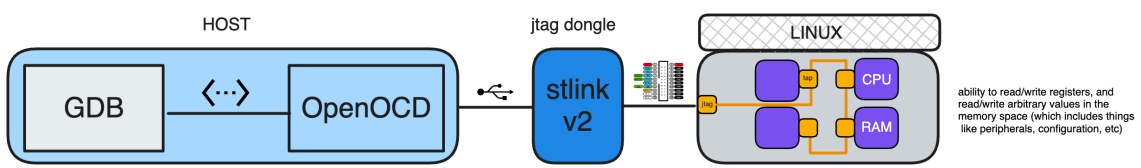
Part C — The kernel under GDB

C · Where GDB attaches

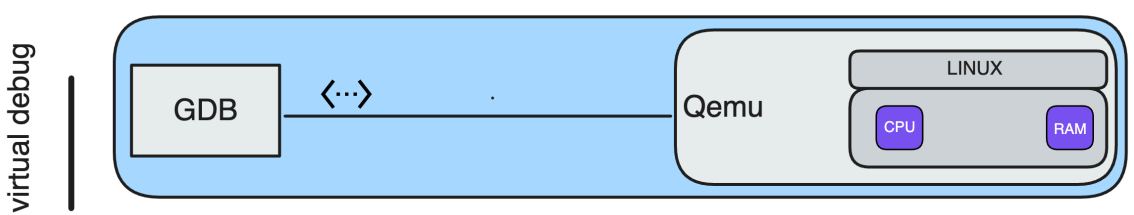
KGDB: a debugger stub inside the kernel talks to GDB over a serial line.



JTAG: a hardware probe controls the CPU; the kernel needs no support.



QEMU (this lab): the emulator’s gdbstub plays the role of the probe.



C · Useful GDB commands

COMMAND	EFFECT
`target remote 127.0.0.1:1234`	attach to QEMU's gdbstub; the whole VM stops
`hbreak <function>`, `hbreak *<address>`	hardware breakpoint; `*` = exact entry address
`continue`, `finish`	resume; run until the selected function returns
`bt`, `frame N`	backtrace; select frame `N`
`p`, `p/x`, `p/d`, `x/s`, `x/4bx`, `x/i`	value; memory as string, bytes, instruction
`lx-ps`, `lx-lsmod`, `p $lx_current()`	kernel helpers: tasks, modules, current task
`lx-symbols <module dir>`	load the symbols of a loaded module
`detach`, `quit`	let the VM run again; leave GDB

The kernel is built with `CONFIG_DEBUG_INFO` (source lines and types) and booted with `nokaslr` (addresses match `vmlinux`). Without the first, GDB shows no line numbers; without the second, frames show as `??`.

C1 · Task: stop the system call that loads your module

Boot a fresh guest with your A2 module, **not loaded**. Use terminal GDB (the browser alternative is at the end of Part C).

QEMU monitor: press `Ctrl-]`, then `c`.

```
gdbserver tcp:127.0.0.1:1234
```

Use the same keys to return to the guest console.

Second host terminal: `make dev-sh`, then

```
cd /sources/linux
make scripts_gdb
gdb -q \
  -iex "add-auto-load-safe-path $PWD" \
  vmlinux
```

GDB

```
set pagination off
target remote 127.0.0.1:1234
p sys_call_table[175]
hbreak __x64_sys_init_module
hbreak do_init_module
continue
```

On x86-64, **175** is `init_module`.

Now run in the QEMU guest

```
insmod /modules/lab-1-work.ko num=7
```

GDB stops before the syscall executes.

C1 · Task: inspect the caller and its arguments

At `__x64_sys_init_module`, print the current task, the syscall number, the three arguments of `init_module(image, length, parameters)` and the first four bytes of the image. `regs` points to the **user registers saved at syscall entry**: `di`, `si`, `dx` hold the three arguments.

```
p $lx_current()->pid
p $lx_current()->comm
p regs->orig_ax
p/x regs->di
p/d regs->si
x/s regs->dx
x/4bx regs->di
```

C1 · Checkpoint

Example from one run (PID, addresses and length vary):

```
$2 = 75
$3 = "insmod\000/0\000\000\000\000\000\000"
$4 = 175
$5 = 0x7f451fb89010
$6 = 228608
0x10c0c90:      " \"num=7\""
0x7f451fb89010: 0x7f    0x45    0x4c    0x46
```

BusyBox adds the spaces and quotes around `num=7`.

Questions.

1. Which task executes this kernel code? Compare with the `comm` printed by `hello_init` in A1.
2. What do the four bytes tell you about what `insmod` hands to the kernel?
3. Why read `regs->di` rather than GDB's `$rdi` at this point?

C2 · Task: from the syscall to your init and back

Continue to `do_init_module`, check which module is being initialized, and return to the syscall entry code one function at a time:

```
continue
p mod->name
bt
finish
finish
finish
p/d $rax
```

On this build the three `finish` commands return from `do_init_module`, `load_module` and `__do_sys_init_module`; check the frame GDB prints after each.

Then `continue` in GDB. In the guest run `echo status=$?`, `cat /proc/modules` and `rmmod module`; load the module again with `num=-1` and repeat the same GDB steps. Finally `detach`, `quit`, and `echo status=$?`.

C2 · Checkpoint

Expected at `do_init_module` : `mod->name` is `"module"` . Selected frames:

```
do_init_module
load_module
__do_sys_init_module
do_syscall_x64
do_syscall_64
entry_SYSCALL_64
```

With `num=7` , each `finish` returns **0** and `$rax` is **0**: status `0` , module `Live` . With `num=-1` , each returns **−22**: status `1` , no module.

Questions.

1. Which function, called by `do_init_module` , finally calls `hello_init` ? (Compare with the call trace in B1.)
2. What is `-22` , and how does it become `invalid parameter` in the guest?
3. Why does the guest console not respond while GDB is stopped?

C3 · Task: stop a timer callback before it runs

The demo module lab-1-list-manip builds a three-node list and arms an `hrtimer`: 3 s after loading, then every second, `my_timer_handler` adds a node and removes the tail. Its traversal contains an **intentional bug**: stop at the callback's first instruction and never let it run. Use a **fresh guest and a fresh GDB session**.

Reuse the monitor and GDB startup commands from C1. In GDB:

```
set pagination off
target remote 127.0.0.1:1234
hbreak do_init_module
continue
```

Guest

```
insmod /modules/lab-1-list-manip.ko
```

GDB at `do_init_module`, before the timer is armed:

```
p mod->name
p $lx_current()->comm
lx-symbols /repo/modules/lab-1-list-manip
p/x &my_timer_handler
hbreak *my_timer_handler
continue
```

Install the breakpoint **before** continuing; the three-second delay is not setup time.

C3 · Checkpoint

At the callback entry

```
bt
p $lx_current()->pid
p $lx_current()->comm
x/i $pc
p/x timer->function
```

Example from one run: PID **0**, name **`swapper/0`** `timer->function` equals the breakpoint address.

End here: exit QEMU (`Ctrl-]` , `x`), then `quit` GDB. Do not `continue` , `detach` or `rmmod` .

Selected frames

```
my_timer_handler
__run_hrtimer
__hrtimer_run_queues
hrtimer_interrupt
__sysvec_apic_timer_interrupt
sysvec_apic_timer_interrupt
```

Questions.

1. Which task is `current` now? Did it ask for the callback? Compare with C1.
2. Why `hbreak *my_timer_handler` , installed at `do_init_module` ?
3. Why does the callback allocate with `GFP_ATOMIC` , and init with `GFP_KERNEL` ?

C4 · Optional task: find the list bug after the panic



In a fresh guest, **without** breakpoints, load `lab-1-list-manip.ko` and let the timer run. The guest stops with `Kernel panic - not syncing: Fatal exception in interrupt`. Then attach GDB and find the faulting instruction, the register that held `cur`, and the values of `cur->data` and `cur->next`.

C4 · Post-fault GDB session

GDB attaches at the panic path, not at the faulting instruction. The original registers are still available through the `regs` parameter of the `page_fault_oops` frame:

```
target remote 127.0.0.1:1234
lx-symbols /repo/modules/lab-1-list-manip
bt
frame N                    # select page_fault_oops; N varies
set $fault = regs
info line *$fault->ip
x/8i $fault->ip-16
p/x $fault->di
p *(struct list_node *)$fault->bx
p/x &head                  # address of the global variable
p/x head                   # pointer to the first node
p *head                    # first node
```

`lx-symbols` gives GDB the name and type of the global `head`. Confirm from the disassembly which register holds `cur` before trusting a saved register. Exit QEMU from the monitor; do not `continue`.

C4 · Checkpoint

Excerpt from one run (condensed, comments added):

```
mov    0x8(%rbx),%rdi
cmpq   $0x0,0x8(%rdi)        # faulting instruction
$1 = 0x0                     # saved RDI
$2 = {data = 1, next = 0x0}  # cur, from saved RBX
```

The oops report shows `CR2: 0000000000000008`. On x86, `CR2` holds the virtual address that caused the latest page fault.

Questions.

1. Why does this null dereference end in a panic, while the one in B1 only killed `insmod`?
2. Why is the faulting address `8` and not `0`?
3. In words, what is wrong with the traversal, and how would you fix it?

Browser frontend: graphical debugging

With the browser frontend, first run the `AOS: Build modules and initramfs` task. Then:

1. run the `AOS: Debug QEMU (1 vCPU)` task;
2. open **Run and Debug**, select `AOS: Attach to QEMU kernel`, and press `F5`;
3. GDB stops in `start_kernel`; press `F5` again to continue booting.

The GDB commands of C1–C4 work in the **Debug Console**. To stop in your module's initialization, enter before loading it:

```
aos-module-symbols /repo/modules/lab-1-work
break hello_init
```

Continue the kernel, then run `insmod /modules/lab-1-work.ko` in the guest. The helper loads the module's symbols at their runtime addresses before `hello_init` executes. GDB, QEMU and the sources all stay inside the browser container.

Wrap-up

Done when

- `modules/lab-1-intro-hello-module` is unchanged; your module is in `modules/lab-1-work`;
- `insmod` with `num=-1` fails with status 1 and leaves no module; with `num=7` it loads, and after writing 9 to the parameter `rmmod module` logs `hello_cleanup: num=9`;
- you can explain each part of the B1 oops report;
- GDB stopped your `insmod` in `__x64_sys_init_module` and `do_init_module`, and you read the caller, the arguments and the return value for `num=7` and `num=-1`;
- GDB stopped at the entry of `my_timer_handler`, and you identified the task it interrupted.

Further reading

- Kernel debugging with QEMU
- Provoking crashes with LKDTM
- Linux kernel debugging
- Debugging the kernel using GDB
- bootlin.com/doc/training/debugging/debugging-slides.pdf
- Serial, KDB and KGDB debugging